

GitHub IO Installation

Create your `yourusername.github.io` repository.

Learning Goals

- Create a repository that GitHub Pages treats as your personal site.
- Sign in to GitHub and push your files with VS Code.
- Publish a first page and open it in the browser.

What You Need

Item	Check
GitHub account	You can sign in at github.com
Git	<code>git --version</code> prints a version
Terminal	Terminal on macOS, or your WSL2 terminal on Windows

You created the account and installed Git in Module 1.

How GitHub Pages Finds Your Site

GitHub Pages gives every account one personal site.

The site comes from a `repository` with a **special name**:

```
yourusername.github.io
```

- We use `yourusername` as your GitHub username for explanation purposes.
- For example, my GitHub username is `nkuase` ; in this case, `yourusername` becomes `nkuase` .

The website address is then:

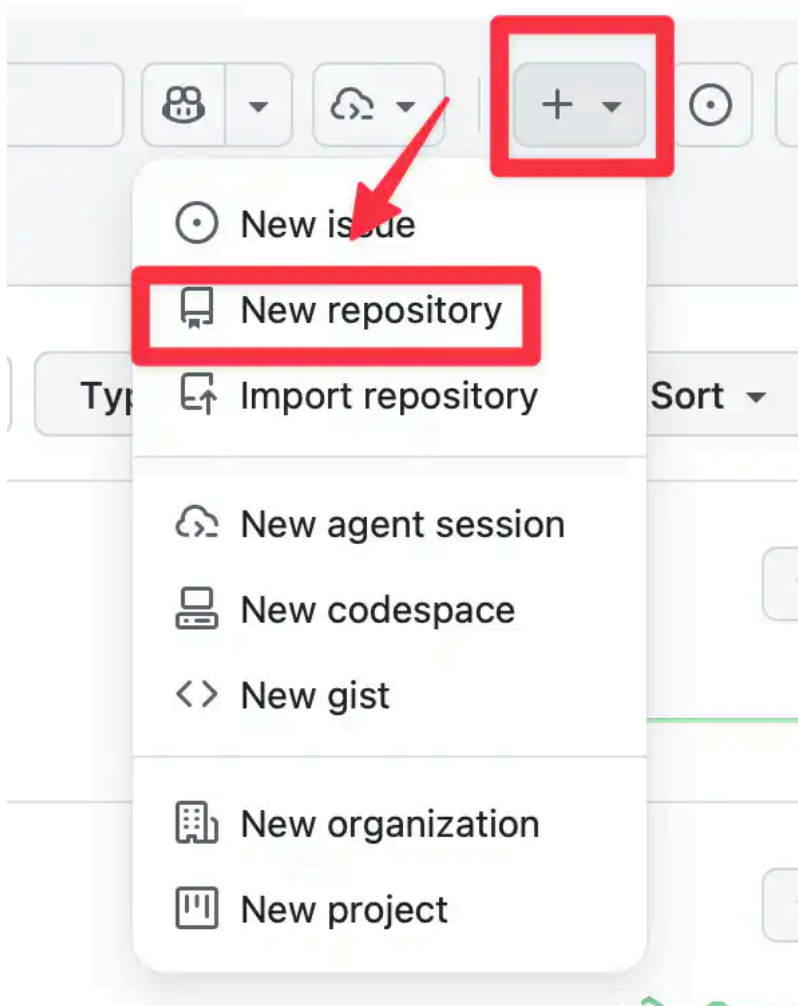
```
https://yourusername.github.io/
```

The name **must match your GitHub username exactly**, written in lowercase (if your username has capital letters, use lowercase in the repository name and URL).

- If it does not match, GitHub Pages will not treat it as your personal site.

Step 1: Create the Repository

1. On GitHub, click + → New repository.



2. Repository name: `yourusername.github.io`
3. Visibility: **Public** (free GitHub Pages requires a public repository).
4. Check **Add a README** file.
5. Click **Create repository**.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General **Use your name instead**

Owner *  nkuase

Repository name *
✓ nkuase.github.io is available.

Great repository names are short and memorable. How about [super-duper-adventure](#)?

Description
0 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository Public

Add README READMEs can be used as longer descriptions. [About READMEs](#) On ☒



Everything you commit is visible to everyone, so never commit passwords, API keys, `.env` files, or personal data.

Step 2: Clone It

Copy the repository URL from the green **Code** button. Then run:

```
git clone https://github.com/yourusername/yourusername.github.io.git  
cd yourusername.github.io
```

```
└─> git clone https://github.com/nkuase/nkuase.github.io.git  
Cloning into 'nkuase.github.io'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.
```

Step 3: Add a First Page

Create `index.html` in the repository folder:

```
<!DOCTYPE html>
<html>
  <head><title>My Site</title></head>
  <body>
    <h1>Hello from GitHub Pages!</h1>
  </body>
</html>
```

GitHub Pages serves `index.html` as the home page.

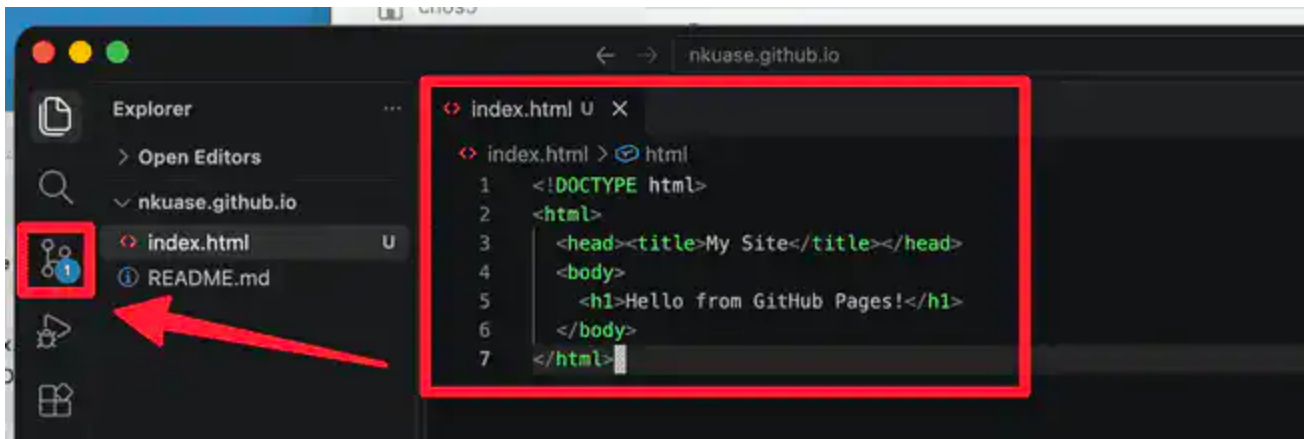
Open the Folder in VS Code

From here, use VS Code. Open the repository folder and create `index.html` there:

```
cd yourusername.github.io  
code .
```

- `code .` opens the current folder in VS Code. Windows users: run it in your WSL2 terminal.
- If `code` is not found, use **File → Open Folder** in VS Code.

- Add the `index.html` using VSCode.
- When a file is added, VSCode shows that a change is made in the repository.



Step 4: Push with VS Code

To push, GitHub must know who you are. This is always required, even for a public repository.

- GitHub does **not** accept your account password in Git commands, so a terminal `git push` fails:

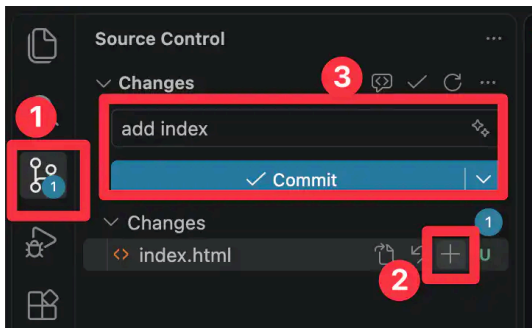
```
remote: Invalid username or token. Password authentication is not supported for Git operations.
```

- Use VS Code to push. It signs you in through the browser, so you need no password or token.

Add and Commit in VS Code

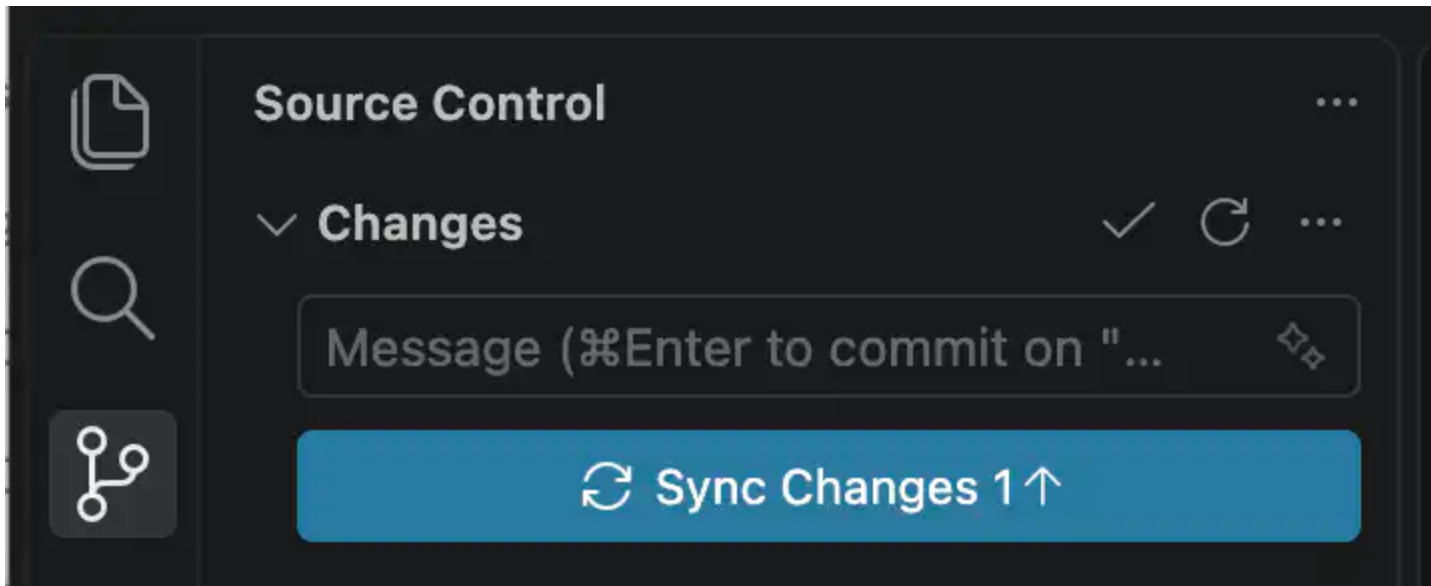
Before pushing to GitHub, you need to add and commit your changes.

1. Select the Git extension.
2. Add the change to the index.
3. Add commit comment and commit.



Push in VS Code

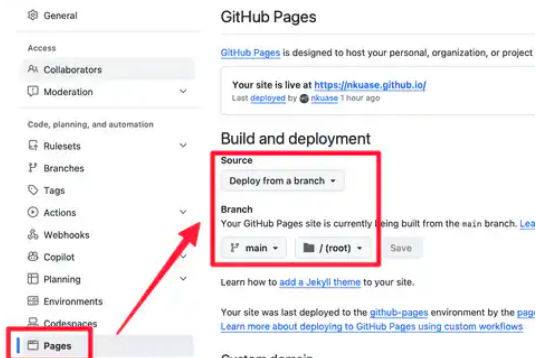
Then, push the changes to the GitHub repository (sync changes); you may need to sign in to your GitHub account to push your changes.



Step 5: Turn On GitHub Pages

In the repository, open **Settings** → **Pages**. Under **Build and deployment**:

- Source: **Deploy from a branch**
- Branch: **main**, folder: **/(root)**
- Click **Save**.



This setting is for the manual practice in lesson 1.

Step 6: Check the Result

Publishing usually takes a few minutes, and sometimes up to 10 minutes. Then open:

```
https://yourusername.github.io/
```

You should see "Hello from GitHub Pages!".

If you see a 404 page, wait a few more minutes and refresh.

Later: Switch to GitHub Actions

For Project 2, your Hugo site is built automatically on every push.

- In **Settings** → **Pages**, change **Source** to **GitHub Actions**.
- Follow lessons 2 to 4 in `3_Github_io`.

Your repository name and URL stay the same.